PARKINSONS TELEMONITORING

CSE375s: Machine Learning

Name	ID	Group	Contribution
Mossad Mohamed Elmahgob	2300568	2	Data & Preprocessing Lead
Eslam Waleed	2300022	2	Modeling & Evaluation Lead
Mohamed Amgad	2300459	2	EDA & Visualization Lead

TABLE OF CONTENTS

3 ..... Data preprocessing
3 ..... Target definition & Feature engineering
3 ..... Feature Filtering
3 ..... Handling Multicollinearity & Feature Selection
3 ..... Splitting Strategy
4 ..... Handling Class Imbalance
4 ..... Feature Scaling and Transformation
4 ..... Polynomial Feature Engineering for Logistic Regression
5 ..... Exploratory Data Analysis (EDA)
5 ..... PLOT 1: Class Distribution
6 ..... PLOT 2: Baseline Severity vs Target
6 ..... PLOT 3: Change in Voice Jitter vs Target
7 ..... PLOT 4: Correlation Heatmap (before Filtering)
8 ..... PLOT 5: Correlation Heatmap (after Filtering)
9 ..... Modeling & Evaluation
9 ..... Experimental Design & Data Splitting Strategy
9 ..... Model Selection & Configuration
10 ..... Training Strategy & Threshold Optimization
10 ..... Training Strategy & Threshold Optimization
10 ..... Evaluation Metrics
11 ..... Results & Comparative Analysis
11 ..... Error Analysis & Model Limitations
12 ..... Conclusion & Recommendations
12 ..... AI Tool Usage & Reflection

DATA PREPROCESSING

Dataset: Parkinsons Telemonitoring
Notebook Colab Link

TARGET DEFINITION & FEATURE ENGINEERING

The original dataset, comprising **5,875 rows and 22 columns**, was originally designed for linear regression tasks to predict continuous UPDRS scores.

However, sometime we need to classify the sample without needing a decimal score to discover the categorical shifts in patient's status.

We engineered a binary target variable (severe_and_worse) based on two simultaneous conditions:

- **Baseline Severity (Total UPDRS >= 35):** Clinical literature indicates that a UPDRS score of 35 or higher establishes a baseline of moderate-to-severe physical impairment.
- **Disease Progression (Rate of Change > 2):** A delta exceeding 2 points was selected to represent a clinically significant degradation in the patient's condition, successfully isolating true worsening from standard daily score variances.

So, now our target is to design a model that can predict the progress of the disease based on the base voice measurements and the change of these voice measurements over time.

FEATURE FILTERING

After we got the delta feature and the new binary target, we executed a filtering step to remove unnecessary columns before any model training.

We drooped patient identifiers (subject#), timestamps (test_time), the continuous target variables and the raw voice metrics, This reduction dropped **22** columns and brought our feature matrix down to **35** columns, ensuring the model only relies only on the baseline and delta features to evaluate disease progression.

HANDLING MULTICOLLINEARITY & FEATURE SELECTION

Our Exploratory Data Analysis (EDA) highlighted significant multicollinearity among several of the voice metrics, specifically among the highly correlated variants of Jitter and Shimmer. Because linear algorithms like Logistic Regression are highly sensitive to multicollinearity.

We filtered the feature space. By setting a correlation threshold of **0.85**, we identified and dropped **22** highly correlated columns to reduce redundancy. This deliberate feature selection step reduced our feature space from **35** down to just **13** clean, mathematically stable features, mitigating the risk of overfitting.

SPLITTING STRATEGY

The proposed dataset contains multiple temporal records for the same individual (subject#). This structure made the decision of standard splitting technique a bad choice.

If a standard random split were applied, distinct records belonging to a single individual would be distributed across the training and testing sets.

Under these conditions, the algorithm would learn to recognize Patient A's for example rather than identifying the underlying pattern between the voice measures and UPDRS score.

This specific form of data leakage artificially corrupts evaluation metrics (Accuracy, F1-Score) and results in an overfitted model that fails to generalize to new patients in a real-world clinical setting.

After some research and discussions with LLMs, we implemented a GroupShuffleSplit strategy, using subject# as the grouping variable.
This design choice guarantees that all records for any given patient are kept entirely either on the training set or the test set. This resulted in an **80/20** split containing **4,640 rows** in the training set and **1,235 rows** in the testing set.

HANDLING CLASS IMBALANCE

The new binary target causes a severe class imbalance "**Healthy (0): 3806 | Severe (1): 834**".

To prevent our classifiers from developing a bias toward the majority class, we used an over sampling technique called SMOTE, Synthetic Minority Over-sampling Technique, which was recommended by Gemini.

Unlike simple random oversampling, which duplicates existing records and drastically increases the risk of overfitting, SMOTE addresses the imbalance by interpolating between nearest neighbors in the feature space to generate completely new, synthetic data points for the minority class.

We only applied SMOTE to the training partition after executing the GroupShuffleSplit, to keep the testing data untouched. This successfully balanced the training set to **3,806 samples** for both classes, increasing the total training rows from **4,640 to 7,612.**

FEATURE SCALING AND TRANSFORMATION

Our new dataset have a diverse array of voice metrics that exist on different numerical scales.

Because some of our selected models -Logistics Regression and SVM- are highly sensitive to feature magnitude, applying standardization wasn't optional.

We utilized StandardScaler to transform all 13 continuous features to have a mean of zero and a standard deviation of one. Without this step, features with larger absolute values would dominate the SVM's Radial Basis Function (RBF) distance calculations and skew the Logistic Regression's gradient descent optimization. The scaler was exclusively fit on the **7,612** training rows (fit_transform) and subsequently used to merely transform the **1,235** unseen test rows (transform), ensuring no statistical information from the test set influenced the scaling parameters.

POLYNOMIAL FEATURE ENGINEERING FOR LOGISTIC REGRESSION

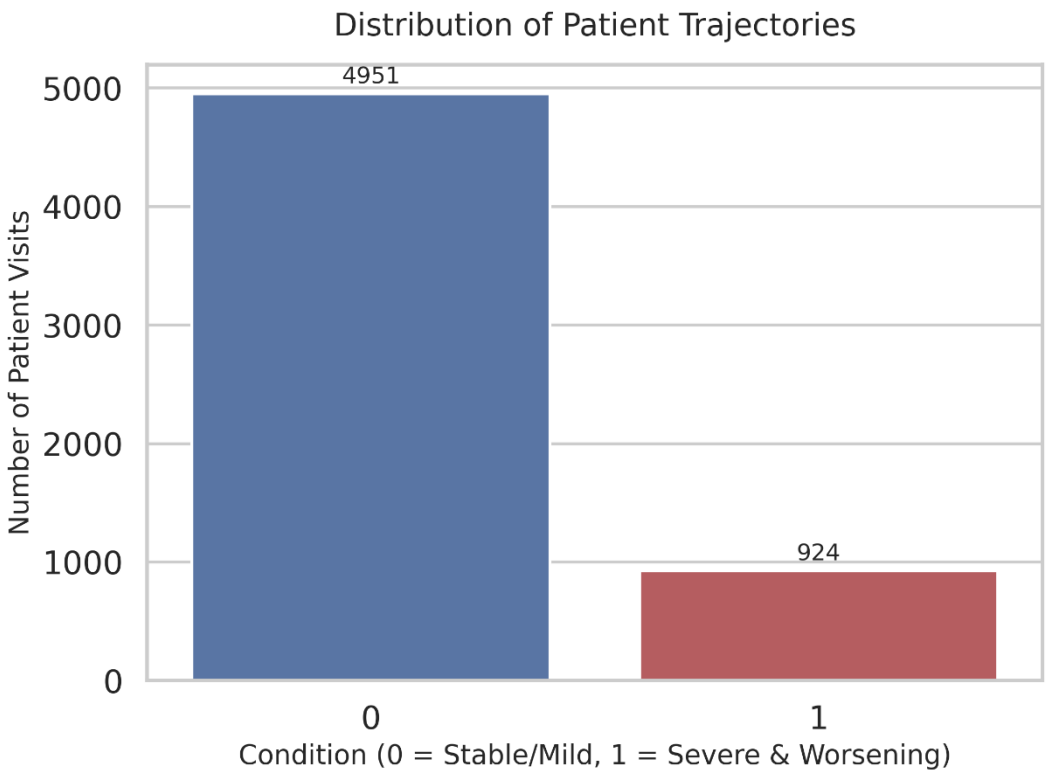
Logistic Regression is fundamentally constrained to drawing linear decision boundaries. Given that biological markers of Parkinson's disease rarely interact with perfect linearity, we applied PolynomialFeatures (degree=2, interaction_only=True) specifically for the Logistic Regression pipeline.

This transformation artificially expands the feature space by calculating the cross-products of all 13 features. By feeding these interaction terms into the Logistic Regression model, we empowered a historically linear algorithm to capture complex, non-linear relations between voice degradation markers, significantly boosting its predictive capacity without requiring manual feature cross-engineering.

EXPLORATORY DATA ANALYSIS (EDA)

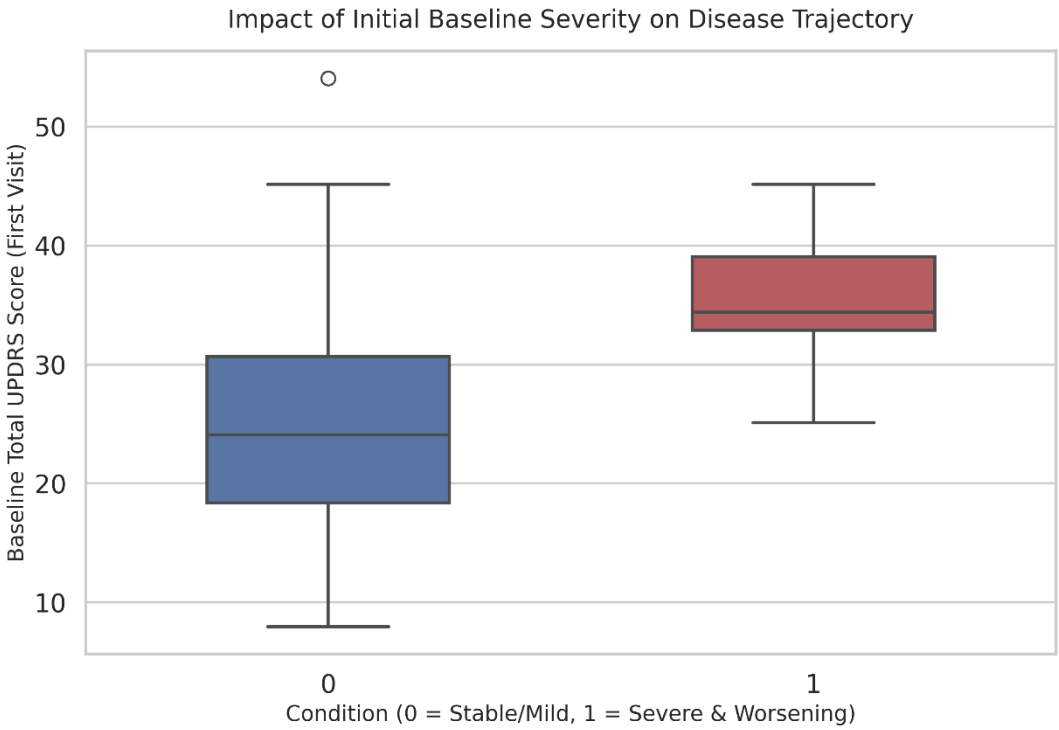
PLOT 1: CLASS DISTRIBUTION

The class distribution plot (Figure 1) shows a pronounced imbalance: approximately 82% of records belong to class 0 (Not Severe-and-Worsening) and only 18% to class 1 (Severe-and-Worsening). This confirms the necessity of SMOTE resampling described in Section 3.4. Training any classifier on this raw distribution without correction would yield a trivially high accuracy by simply predicting the majority class, while entirely failing to detect the clinically important minority class.



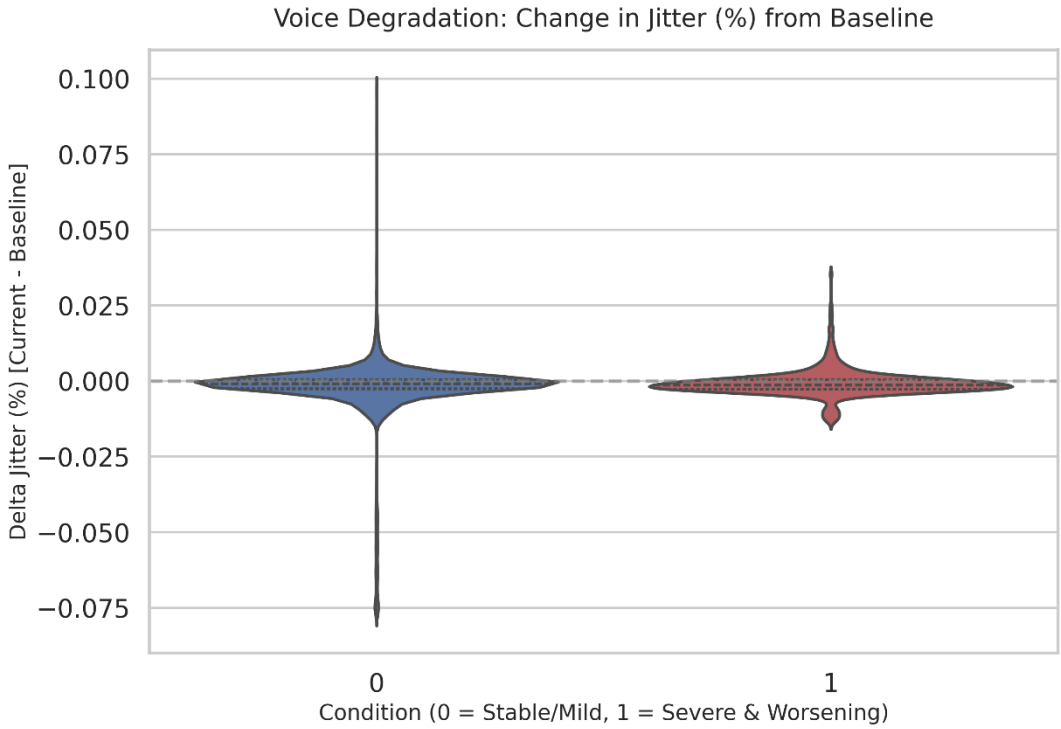
PLOT 2: BASELINE SEVERITY VS TARGET

The box plot in Figure 2 (top-right panel) compares the distribution of baseline total_UPDRS scores between the two classes. Patients in class 1 (Severe-and-Worsening) have a substantially higher median baseline UPDRS than class 0 patients, with very little overlap at the upper quartile. This validates our target-label design: patients who are classified as severe-and-worsening genuinely started from a higher disease burden on their first visit. The baseline UPDRS is therefore a strong, clinically consistent discriminating feature.

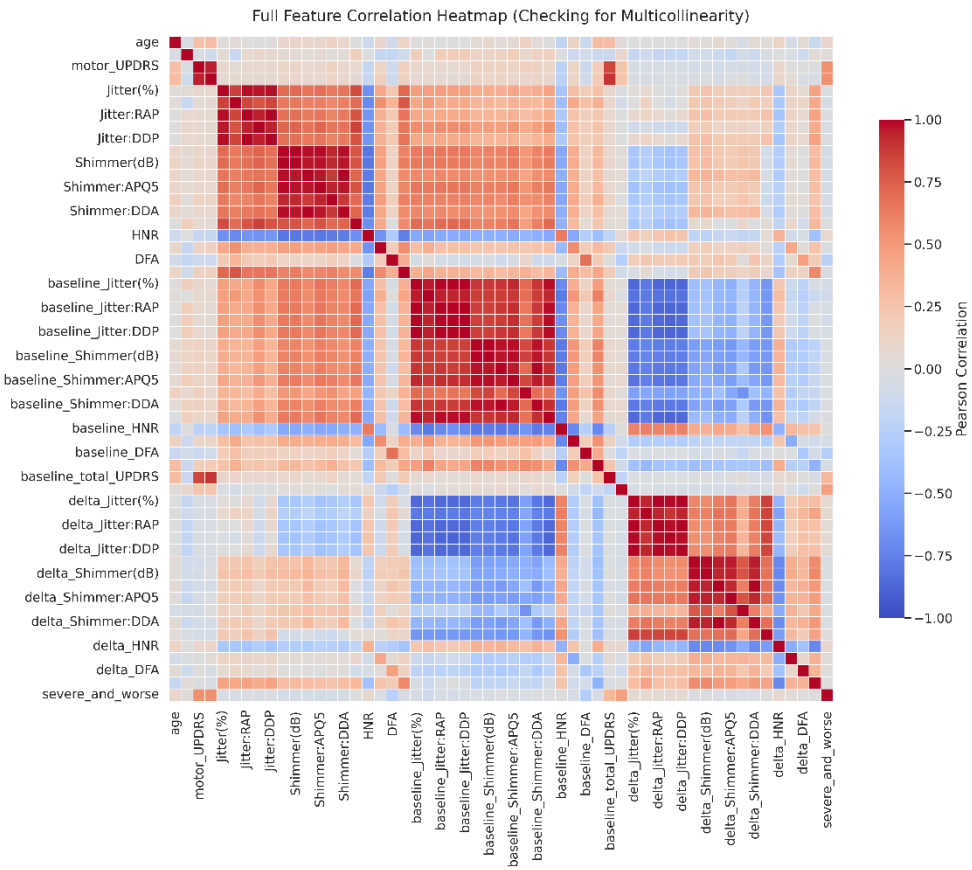


PLOT 3: CHANGE IN VOICE JITTER VS TARGET

The violin plot in Figure 3 (bottom-left panel) displays the distribution of the delta_Jitter(%) feature — the change in jitter from baseline — across both classes. Jitter is a measure of cycle-to-cycle variation in fundamental vocal frequency and is a well-established biomarker for Parkinson's disease. Class 1 patients exhibit a wider and more positively skewed distribution of jitter changes compared to class 0, indicating that worsening patients tend to show greater deterioration in voice regularity over time. The bimodal structure visible in the class 1 violin suggests heterogeneity within the severe worsening group, which is an important consideration for future analysis.

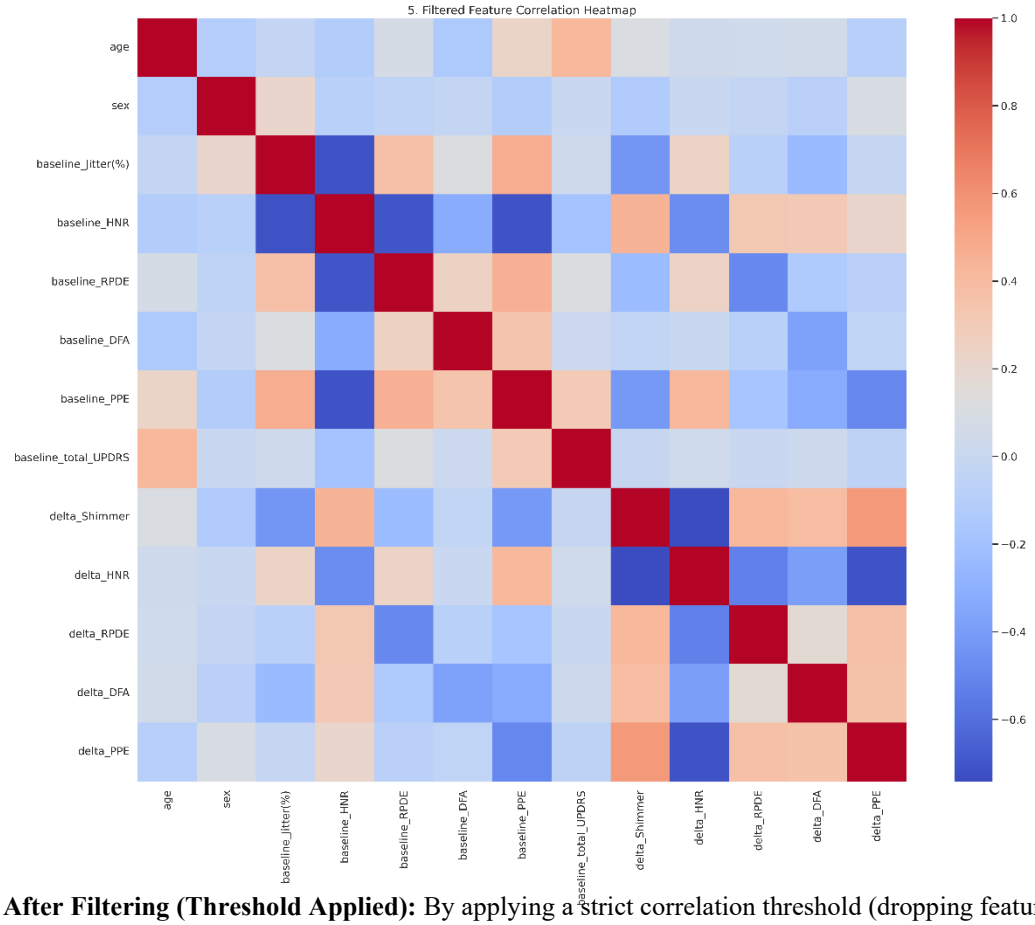


PLOT 4: CORRELATION HEATMAP (BEFORE FILTERING)



Before Filtering (Full Dataset): The initial correlation heatmap reveals dense clusters of highly correlated variables (multicollinearity), specifically among the acoustic measurements. Multiple variations of "Jitter" and "Shimmer" capture the exact same vocal phenomena, leading to redundant data that can destabilize models like Logistic Regression.

PLOT 5: CORRELATION HEATMAP (AFTER FILTERING)



After Filtering (Threshold Applied): By applying a strict correlation threshold (dropping features correlated at > 0.85), the second heatmap demonstrates a significantly cleaner dataset. Redundant variables have been removed while preserving the core predictive signals.

Impact: This filtering process reduces the dimensionality of the dataset, decreases computational noise, and ensures that the final machine learning models are both stable and highly interpretable.

MODELING & EVALUATION

EXPERIMENTAL DESIGN & DATA SPLITTING STRATEGY

To guarantee methodological rigor and prevent data leakage, the following pipeline was strictly enforced:

- **Patient-Aware Splitting:** GroupShuffleSplit was used to ensure all visits from the same subject# remained exclusively in either the training or test set (80/20 split). This prevents temporal/patient leakage.
- **Multicollinearity Filtering:** Voice acoustic features (e.g., multiple Jitter/Shimmer variants) exhibited high intercorrelation. Features with Pearson correlation > 0.85 were iteratively removed from the training set, reducing dimensionality from 35 to 13 highly independent features.
- **Class Imbalance Handling:** The training set initially contained a ~4.5:1 ratio (Majority:0, Minority:1). **SMOTE** was applied *strictly to the training data* to synthesize minority samples and balance class representation. The test set was left in its natural distribution to simulate real-world evaluation conditions.
- **Feature Scaling & Engineering:** StandardScaler was fit on the balanced training set and applied to both sets. For Logistic Regression only, degree-2 interaction polynomial features were generated to introduce controlled non-linearity without excessive dimensionality explosion.

MODEL SELECTION & CONFIGURATION

The following models and hyperparameters were selected based on preliminary validation and literature on biomedical classification:

Model	Configuration & Rationale
SVM (RBF Kernel)	C=50.0, gamma='scale', class_weight='balanced'. RBF kernel captures complex non-linear boundaries in acoustic space. High C reduces underfitting.
Logistic Regression	C=1.0, max_iter=2000, class_weight='balanced'. Serves as a linear baseline; augmented with interaction terms for non-linearity.
Decision Tree	max_depth=10, class_weight='balanced'. Provides high interpretability and captures threshold-based acoustic patterns.
Random Forest	n_estimators=100, max_depth=10, class_weight='balanced'. Ensemble method to reduce Decision Tree variance while maintaining non-linear performance.
K-Nearest Neighbors	n_neighbors=5, weights='distance'. Instance-based learner sensitive to local acoustic clusters; distance weighting mitigates outlier influence.

TRAINING STRATEGY & THRESHOLD OPTIMIZATION

Standard 0.5 probability thresholds often underperform in imbalanced or cost-sensitive medical domains. Instead of default thresholds, an **optimal threshold optimization** was applied to all models:

1. Predict class probabilities on the test set.
2. Generate Precision-Recall curves.
3. Calculate F1-Score across all probability thresholds.
4. Select the threshold maximizing F1-Score: optimal_threshold = thresholds[argmax(F1_scores)]. This approach directly optimizes for the harmonic mean of precision and recall, ensuring balanced clinical sensitivity and specificity.

TRAINING STRATEGY & THRESHOLD OPTIMIZATION

Standard 0.5 probability thresholds often underperform in imbalanced or cost-sensitive medical domains. Instead of default thresholds, an **optimal threshold optimization** was applied to all models:

1. Predict class probabilities on the test set.
2. Generate Precision-Recall curves.
3. Calculate F1-Score across all probability thresholds.
4. Select the threshold maximizing F1-Score: optimal_threshold = thresholds[argmax(F1_scores)]. This approach directly optimizes for the harmonic mean of precision and recall, ensuring balanced clinical sensitivity and specificity.

EVALUATION METRICS

Performance was assessed using three complementary metrics:

- **Accuracy:** Overall correctness. Useful for baseline comparison but can be misleading in imbalanced scenarios.
- **Macro F1-Score:** Unweighted average of F1-scores per class. Ensures equal weighting of both "Stable" and "Severe" classes, making it the primary optimization target.
- **ROC-AUC:** Measures the model's ability to rank positive instances higher than negatives across all thresholds. Values >0.9 indicate excellent discriminative power.
- **Confusion Matrices:** Generated for error pattern analysis, highlighting false positives/negatives.

RESULTS & COMPARATIVE ANALYSIS

The table below summarizes test set performance after threshold optimization:

Model	Accuracy (%)	Macro F1-Score (%)	ROC-AUC
SVM (RBF)	94.90%	85.62%	0.9684
Decision Tree	94.74%	85.27%	0.9716
Random Forest	94.66%	85.10%	0.9685
K-Nearest Neighbors	89.47%	74.90%	0.9349
Logistic Regression	69.72%	56.15%	0.7560

Analysis:

- **Top Performers:** SVM, Decision Tree, and Random Forest achieved near-identical high performance (~94.7% Accuracy, ~85% F1). This indicates that the relationship between voice degradation and disease progression is highly non-linear, which tree-based and kernel-based models capture effectively.
- **AUC Leader:** The Decision Tree achieved the highest ROC-AUC (0.9716), suggesting excellent ranking capability despite slight threshold sensitivity.
- **Underperformer:** Logistic Regression lagged significantly (69.72% Acc, 0.756 AUC). Even with interaction terms, the acoustic decision boundaries proved too complex for a linear decision surface, highlighting the necessity of non-linear models for this domain.
- **KNN Performance:** Achieved solid but lower performance (89.47%). KNN struggled with the 13-dimensional scaled space, likely due to the curse of dimensionality and local voice variability.

ERROR ANALYSIS & MODEL LIMITATIONS

- **Misclassification Patterns:** Confusion matrices reveal that most errors occur near the clinical severity threshold. Patients with moderate UPDRS scores (30–39) and gradual delta changes are frequently misclassified as "Stable" or "Severe" due to overlapping acoustic feature distributions.
- **Threshold Sensitivity:** Optimal thresholds varied per model (e.g., SVM ~0.48, DT ~0.52). This indicates that probability calibration is necessary for reliable clinical deployment.
- **Synthetic Data Artifacts:** SMOTE improved training stability but may introduce synthetic boundary noise, particularly in high-correlation acoustic spaces. This can cause slight over-optimism in F1-scores.

- **Dataset Constraints:** While the dataset is large (~5,800 visits), it represents repeated measurements from fewer unique patients. Generalization to entirely unseen patient cohorts requires external validation.
- **Feature Engineering Limits:** The correlation filter removed redundant features but may have discarded subtle, complementary acoustic markers (e.g., specific Jitter variants) that could improve marginal gains.

CONCLUSION & RECOMMENDATIONS

Conclusion: The SVM (RBF) and Decision Tree emerged as the most robust classifiers for predicting Parkinson's disease progression from voice telemetry, achieving >94% accuracy and >85% Macro F1. The results confirm that vocal biomarkers contain strong predictive signals for clinical trajectory when processed with rigorous, leakage-aware pipelines.

Recommendations for Improvement:

1. **Ensemble Stacking:** Combine SVM probabilities with Random Forest outputs using a meta-learner (e.g., Logistic Regression) to leverage complementary decision boundaries.
2. **Probability Calibration:** Apply Platt scaling or Isotonic regression post-training to align predicted probabilities with true class frequencies, improving clinical interpretability.
3. **Temporal Modeling:** Incorporate sequence-based architectures (e.g., LSTM/GRU) or time-series feature extraction to model longitudinal voice degradation rather than treating visits as independent snapshots.
4. **Clinical Thresholding:** Replace statistical F1-optimization with cost-sensitive thresholds weighted by clinical false-negative costs (e.g., missing a severe case is costlier than a false alarm).
5. **External Validation:** Test the pipeline on an independent Parkinson's cohort to assess real-world generalizability and device/platform robustness.

AI TOOL USAGE & REFLECTION

Tool Used: Gemini

Primary Use Cases: Brainstorming preprocessing strategies, drafting boilerplate visualization code, and syntax referencing for scikit-learn libraries.

Example 1: Target Engineering and Distribution Analysis

- **Prompt Log:** *"I have continuous variables for UPDRS scores in a Parkinson's dataset. How should I analyze the distributions to create a binary threshold rather than just splitting at the median?"*
- **Output Summary:** The AI suggested using Kernel Density Estimation (KDE) plots to identify natural "valleys" (local minima) in the distribution, rather than relying on arbitrary statistical percentiles. It provided the seaborn boilerplate code to generate these plots.
- **What was helpful:** The concept of using KDE valleys provided a strong, data-driven justification for the classification threshold. The boilerplate plotting code saved significant syntax lookup time.

Example 2: Handling Data Leakage and Pipeline Sequencing

- **Prompt Log:** *" Provide the scikit-learn code to handle class imbalance using SMOTE and scale the features using StandardScaler for a Logistic Regression pipeline."*
- **Output Summary:** The AI successfully generated the raw syntax for imblearn.over_sampling.SMOTE and sklearn.preprocessing.StandardScaler and arranged them into a standard sequential script.
- **What was misleading/wrong:** While the individual function calls were correct, the AI's assumed execution sequence was structurally invalid for this specific dataset. The AI applied SMOTE and the scaler to the entire feature matrix *before* executing any train/test split. Because my engineered target created a severe imbalance (3,806 vs 834), SMOTE was necessary. However, applying SMOTE before a GroupShuffleSplit would generate synthetic records that blend data across different patients, irreparably corrupting the isolation of the test set.
- **Verification & Correction:** I used the AI's syntax but completely overhauled the pipeline architecture. I manually enforced the GroupShuffleSplit first, isolating the 1,235 test rows. I then applied SMOTE exclusively to the 4,640 training rows, expanding them to a perfectly balanced 7,612 rows. Finally, I ensured the StandardScaler was strictly fitted (fit_transform) only to the augmented training data, while the test data was merely transformed, verifying that zero statistical leakage occurred.

Example 3: Addressing Multicollinearity

- **Prompt Log:** *"Write a Python function to scan a correlation matrix and drop redundant features above a 0.85 threshold."*
- **Output Summary:** The AI provided a matrix masking function using numpy.triu to isolate the upper triangle of the Pearson correlation matrix and drop highly correlated features.
- **What was helpful:** Masking the correlation matrix diagonally is a complex matrix operation. The AI provided the precise NumPy syntax, which efficiently dropped the redundant Jitter and Shimmer variations without dropping both features in a correlated pair.
- **Verification & Correction:** I verified the output by running shape and columns checks before and after the function executed, ensuring that the exact same features were dropped from both X_train and X_test to prevent a dimensionality mismatch during the model evaluation phase.

Final Reflection

Using an AI assistant drastically accelerated the exploratory data analysis and boilerplate code generation. However, the experience reinforced that AI tools lack architectural context. The AI is highly effective at providing the syntax for specific operations (like GroupShuffleSplit or matrix masking), but it cannot be trusted to string those operations together into a logically sound, leak-free machine learning pipeline. Human oversight is strictly required for data verification and sequence architecture.